Course on Knowledge Graphs in the era of Large Language Models

Laboratory 6: Knowledge Graphs and Language Models

Ernesto Jiménez-Ruiz

March 26, 2026

Contents

1 Git Repository

Support codes for the laboratory sessions are available in *github*. There is always a `requirements.txt` within each lab folder. It is recommended to use environments, but it is not strictly necessary.

`https://github.com/city-knowledge-graphs/kg-course-valgrai`

2 Using Large Language Models

There is a plethora of available Large Language Models, with different sizes and modes. OpenRouter (`https://openrouter.ai/`) serves as a unified interface to access LLMs. In the GitHub repository, I have added example code in Python (including a Jupyter Notebook). OpenRouter is very simple use, and one just needs to indicate the model to be used and the prompt. The example codes use the *OpenAI API*. I have created an OpenRouter API key (will be shared via Teams). Add the *openrouter.key* in the 'files' folder within the Python repository. If you use a different location, you will need to update the example codes. Please do not make the API key public (this includes commits to public GitHub repositories), otherwise the key will become invalid.

Task 1 Modify the prompts in the example codes and test that the code works.

Tip 1: *Some LLMs can be expensive when invoked multiple times, so one may need to limit the number of requests to solve a given task.*

Tip 2: *The current key contains $15 for all students, but one could create additional keys and add additional credits.*

OpenRouter provides extensive documentation on the use of the API and the available models:

- API: `https://openrouter.ai/docs/quickstart`
- Available models and cost: `https://openrouter.ai/models?fmt=cards&input_modalities=text`

For non-programmers: you can also access OpenRouter via its chat: `https://openrouter.ai/chat`

3 Using OWL2Vec*

OWL2Vec* is a system that embeds the semantics of an OWL ontology.[1] OWL2Vec* currently relies on random walks and word embedding and learns a (numerical) vector representation for each URI and word in the ontology. OWL2Vec* generates a document of sentences from the ontology (using this RDF2vec implementation: `https:`

[1] OWL2Vec*: Embedding of OWL Ontologies. Machine Learning Journal (2021). [Journal version], [arXiv version], [GitHub].

//github.com/IBCNServices/pyRDF2Vec/) and then applies Word2vec as neural language model (see details here: https://radimrehurek.com/gensim/models/word2vec.html).

3.1 Installation

Please use the the OWL2Vec* version (zip file) from the GitHub repositoriy (OWL2Vec-Star-IN3067-INM713.zip). Running OWL2Vec* is very easy. Before you start, *(i)* download OWL2Vec* (OWL2Vec-Star-IN3067-INM713.zip), *(ii)* unzip the file, and *(iii)* open the folder where the codes are (*e.g.*, location of setup.py).
Tip: *To run the commands in Options 1 and 2 below make sure you are located in the right directory, also to run the jupyter notebook that is provided within the zip file.*
Tip 2: (Windows users)You may also need to install https://visualstudio.microsoft.com/visual-cpp-build-tools/ and follow these instructions.

Option 1 (running OWL2Vec* from the terminal):

1. Install Python 3 (if not done before): https://www.python.org/downloads/
2. Install setuptools: https://pypi.org/project/setuptools/
   e.g., `pip install setuptools`
3. Run this command in the terminal:[2] `make install` or `python setup.py install` (in Windows and other distributions). You may need Root privileges in Linux.
4. Run OWL2Vec* in the terminal:
   `owl2vec_star standalone --config_file default.cfg`

Option 2 (running OWL2Vec* from a notebook or a python script (**recommended**)):

1. Install Python 3 (if not done before): https://www.python.org/downloads/
2. Install `pip` (if not done before): (https://pip.pypa.io/en/stable/installation/)
3. Install the library dependencies in the file `requirements_owl2vec.txt` (**use of environments is strongly recommended, see Lab 1**):
   e.g., `pip3 install -r requirements_owl2vec.txt`
4. Run the notebook: `jupyter_notebook_owl2vec.ipynb`

[2]For Windows user you need to use the Windows Command Prompt. I can help on this: https://www.makeuseof.com/tag/a-beginners-guide-to-the-windows-command-line/

3.2 Configuration

In the file `default.cfg` we can indicate the following configuration parameters for OWL2Vec* (see lecture slides or the OWL2Vec* paper for more details):

- `ontology_file`: input OWL ontology.
- `embedding_dir`: path and file name for the output embeddings.
- `cache_dir`: path to store intermediate files.
- `ontology_projection`: if the graph projection of the ontology is enabled.
- `projection_only_taxonomy`: if only rdfs:subClassOf is taken into account.
- `multiple_labels`: if more than the main label is considered.
- `avoid_owl_constructs`: if OWL 2 constructs are avoided in the generated document.
- **Walker parameters:** to build the document sentences.
  - `walker`: random walks or Weisfeiler-Lehman (wl) subtree kernel.
  - `walk_depth`: depth of each of the performed walks to create each sentence.
- **OWL2Vec* parameters:** type of document of sentences to build. One could create a document with only words (*i.e.*, Lit_Doc)
  - `URI_Doc`: sentences with only entity URIs.
  - `Lit_Doc`: sentences with only words.
  - `Mix_Doc`: mixing words and URIs in the sentences.
  - `Mix_Type`: the type for generating the mixture document (all or random).
- `pre_train_model`: optional path to a pre-trained Word2vec model.
- **Word2vec parameters:**
  - `embed_size`: the size for embedding.
  - `iteration`: number of iterations in training the language model.
  - `min_count`: minimum word occurrence to create an embedding.
  - `window`, `negative` and `seed`: other Word2vec training paramaters.

3.3 Output

OWL2Vec* produces two embedding files as output (in `embedding_dir`):

- Gensim model (`.embedding` file).
- Word2vec text format (`.txt` file). This file is readable with a text editor. Each line is composed by a key (*i.e.*, word) and a value (*i.e.*, vector).

The generated sentence document is available in the `cache` output folder (*i.e.*, `document_sentences.txt`). This file is interesting to see how the ontology has been transformed into a text document.

3.4 Using the Embeddings

The computed embeddings can be used to calculate (cosine) similarities, perform clustering of entities, and use them in subsequent machine learning tasks.

Python. In the GitHub repository, there is an example script (`load_embeddings.py`) that loads pre-computed embeddings, in this case by OWL2Vec*. In python we use the gensim `keyedvectors` library: `https://radimrehurek.com/gensim/models/keyedvectors.html`. Once the model has been loaded one can calculate (cosine) similarities among ontology entities/words (*i.e.*, using their associated vectors) and get their closest entities/words. The codes are also provided within the jupyter notebook.

3.4.1 Embedding the Pizza ontology

The ontology is available in the GitHub repositories.

Task 2 Run OWL2Vec* over the `pizza.owl` ontology.

Task 3 Compute the similarity between the following elements.

- `http://www.co-ode.org/ontologies/pizza/pizza.owl#Margherita` and `margherita`
- `http://www.co-ode.org/ontologies/pizza/pizza.owl#Hot` and `http://www.co-ode.org/ontologies/pizza/pizza.owl#Medium`
- `CheesyPizza` and `CheeseTopping`

Task 4 Get the most similar entities/words for the following elements:

- `Hot`
- `http://www.co-ode.org/ontologies/pizza/pizza.owl#CheesyPizza`
- `http://www.co-ode.org/ontologies/pizza/pizza.owl#Fiorentina`
- `Soho`

Task 5 (Optional) Perform clustering using the K-means algorithm for example and visually represent the results in 2-dimensions using PCA.

Python, relevant references:

- `https://jakevdp.github.io/PythonDataScienceHandbook/05.11-k-means.html`
- `https://jakevdp.github.io/PythonDataScienceHandbook/05.09-principal-component-analysis.html`

3.4.2 Embedding the FoodOn ontology

In the following tasks we will use teh `FoodOn` ontology (`https://foodon.org/`). The ontology is also available in the GitHub repositories.

Task 6 Run OWL2Vec* over the `foodon-merged.owl` ontology. This ontology is larger and computing the embeddings will take much longer. The embeddings will also be richer as the generated document is rather large (>1Gb).

Task 7 Compute the similarity between the following elements.

- `http://purl.obolibrary.org/obo/FOODON_00002434` (mushroom food product) and `mushroom`
- `http://purl.obolibrary.org/obo/FOODON_00002434` (mushroom food product) and `http://purl.obolibrary.org/obo/FOODON_00001287` (mushroom food source)
- `http://purl.obolibrary.org/obo/FOODON_03304544` (frozen chicken) and `http://purl.obolibrary.org/obo/FOODON_03311876` (baked chicken)

Task 8 Get the most similar entities/words for the following elements:

- `mushrooms`
- `chicken`
- `http://purl.obolibrary.org/obo/FOODON_03411323` (rabbit)
- `http://purl.obolibrary.org/obo/FOODON_03411129` (trout and salmon family)

Task 9 (Optional) Perform clustering using the K-means algorithm for example and visually represent the results in 2-dimensions.